

Lab 1: Preliminaries

Instructor: Santiago Torres-Arias

This document is big and may look daunting, but it's a lot of hand-holding! I wanted to err on the side of verbosity!!

1 Lab information and Goals

During this lab we will do a “test run” of how the labs are going to be delivered and carried out. We will also use the opportunity to set up an experiment environment that we will use throughout the rest of the course.

1.1 Deliverable format

Formatting does not matter, use whatever you feel comfortable, but pay close attention to presentation. Consider the following aspects in your report:

- Think about what you *need to say and why*: that is, you want to show me you did the activities successfully. How would you prove that?
- Think about the *audience* and the *level of depth*: who is reading this? and did you say enough to say what you wanted to say? is anything missing? is the background needed too much? too little?

With this aside, the rest is pretty much free for all.

1.2 Grading

Sections in the lab will be labeled with a series of points, and the following will be considered:

- Did you do it successfully?
- Did you understand why? or did you just stumble to the solution?
- Is the methodology and results properly presented?

In the document, you will find boxes like this

some questions or things I want

Points: 1

Use these to guide the things that should be included in your report.

I don't expect you to be a professional or weathered researcher at the time of preparing your reports. As a consequence, I'll often give you feedback for the first time, and not discount any points because of that. Further, if you don't pay closer attention to it in future iterations then I will go ahead and discount points.

I care more about you learning and improving than having perfect reports! Ask questions! let's do cool things!

1.3 Goals

With this aside, in this lab we will focus on setting up an ARM cross compiler, as well as doing a little bit of investigation on how to:

1. Build and study built binaries
2. Emulate and run the built binaries
3. Debug and dissect the binaries

If we are successful – and we will be – we should have a little binary that runs and lets us study itself in the right environment.

1.4 Motivating example

You may have run into this screen when booting a linux system

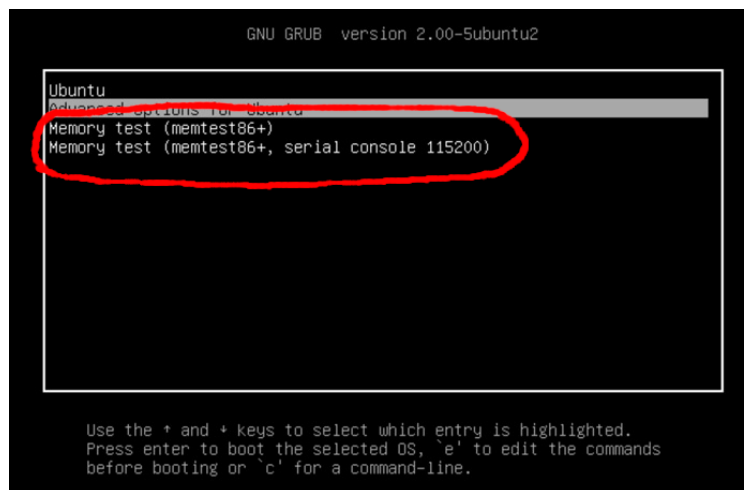


Figure 1: A GRUB bootloader showing boot options (including memcheck)

Circled in red, you may notice an option called “memory test.” You may not have run it (because who runs into memory errors nowadays), but it runs a program called memtest86+. According to wikipedia, this program does the following:

MemTest86 and Memtest86+ are memory test software programs designed to test and stress test an x86 architecture computer’s random-access memory (RAM) for errors, by writing test patterns to most memory addresses, reading back the data, and comparing for errors.[5] Each tries to verify that the RAM will accept and correctly retain arbitrary patterns of data written to it, that there are no errors where different bits of memory interact, and that there are no conflicts between memory addresses.

Memtest is a “bootable image” that comes in many flavors (we will learn about those later), which is to say, it doesn’t require any sort of setup from any intermediate bootloaders (other than, in this case, GRUB [3]). Let’s use this as a motivation for a very small program, that we will write in assembly, that will run through memory in the system and “do stuff.” This time around we don’t care **about the specific tests, but about running something from bare-metal.**

Ok, let’s get started.

2 Installing the required tools

We need to install “three” major tools/toolchains in order to get started.

1. arm-none-eabi-gcc
2. arm-none-eabi-binutils
3. qemu-system-arm

I use quotation marks over three because this may be split or merged and you may end up installing 2, 4 or even 5 different tools rather than 3. To elaborate, *I don't care what OS you use, but rather that you understand and test that your setup works*. Philosophically, I believe that forcing you down any path (e.g., I use Arch¹) is not entirely useful to understand why things work and don't work. All in all, getting stuck setting up your tools is part of the experience.

2.1 arm-none-eabi-gcc and arm-none-eabi-binutils

There are fundamentally two ways to get this (and any of the packages above for that matter). First, you can use your package manager of choice to install it. Second, you can build it from source yourself. I don't want you to spend a lot of time on this (and gcc does take a bit to compile), so I suggest you go ahead and use your package manager. A handy website to check this in a cross OS way is to use pkgs.org:

<https://pkgs.org/download/arm-none-eabi-gcc>

This list may not include your OS, this is because they may have named it differently. For example, Ubuntu uses a name closer to `gcc-arm-none-eabi`. It's unfortunate, but these names follow local conventions by the maintainers of each individual OS. If you really can't find it, reach out to me and we can figure (i.e., google) it out together.

what is included in this installation? what tools and libraries are being included?	Points: 1
---	------------------

For example, I can run a command such as `pacman -Qlq arm-none-eabi-gcc` and inspect the contents of the package. Then I'd look up what do some of the files do (it's a lot of them) to give me an understanding of this.

2.2 qemu-system-arm

Then, we need a place to run these new binaries. To do so, we will use Qemu [2], which is a very bare-bones, easy to setup hypervisor (we will learn about those later in class). With it, we will be both able to emulate a CPU or a full fledged board (and we will do both today).

You can follow the steps above to set up `qemu-system-arm`, but you may require further tools. Ideally, we want to test that this runs without figuring out whether our code is running/working, so let's use something that's known to work on the system.

2.2.1 qemu-arm

First, we will test the CPU emulator, `qemu-arm`, this is a little simpler (because you don't need to think about anything other than the CPU). Prepare some simple program².

```
# qemu-arm cpu-emu-test
hello world!
#
```

Not very exciting right? well try running this outside of `qemu-arm` and it should fail. If this works, we can move on to simulating a whole system.

¹btw

²there should be one in brighspace called `cpu-emu-test`

2.3 qemu-system-arm

As opposed to the above, `qemu-system-arm` does quite a bit more. Instead of just emulating the CPU, it will *actually* emulate a machine. To do so, it'll set up a bunch of hardware features and set everything up for programs to interact with. We can test our installation works by doing the following:

```
# qemu-system-arm -machine xilinx-zynq-a9 ...
```

If everything is correct, something like Figure 2 will pop out.

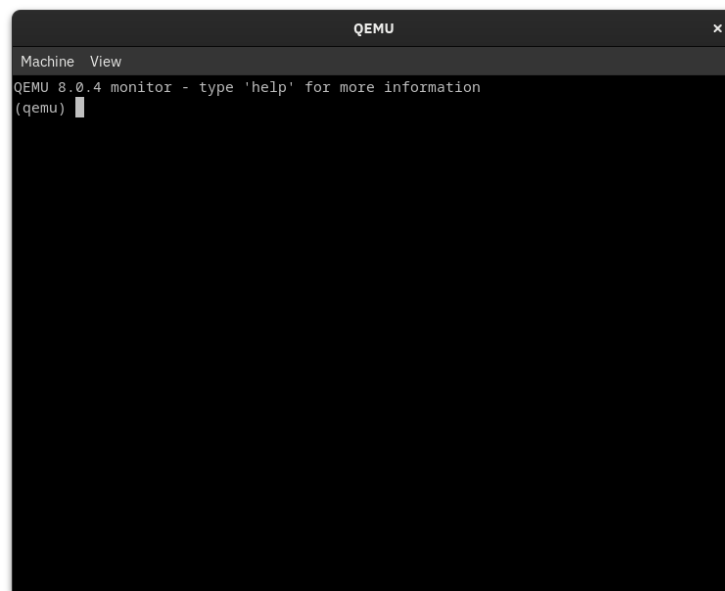


Figure 2: A window of a successful qemu machine running

This may not be super exciting, but this really means that your machine is simulating a xilinx fpga-soc board (a Zynq a9). I chose this at random, but you should take a look at `qemu-system-arm -machine help` and see what options there are. If you don't have the zynq-a9 on your machine you can really try any other machine. We're just testing things out after all.

Show me that you set both things up, feel free to elaborate your thoughts about why/how things work after reading a bit on both **Points: 2**

3 Building ARM binaries

Ok, so we have the tools we need (we may need another one but let's not get ahead of ourselves). Let's build something that we can run in both of these. Since I want us to start at the very bottom of things, I don't want you to focus on a hello world, but something even simpler. Take a look at the snippet below:

```
.global _start
_start:
top:
    eor r1, r1
    b top
```

Copy paste this code in a file of your choice, say, `babysteps.s` and save it. We will compile it with the following commands:

```
arm-none-eabi-as babysteps.s -o babysteps.o
arm-none-eabi-ld -g babysteps.o -o babysteps.elf
```

This should produce an executable file. We compiled it with debugging symbols, and this will be useful a little bit later.

Explain what this assembly code does. Explain what did we just do. Explain what each command does, as well as each file and flag passed to it	Points: 1
---	------------------

3.1 Studying the compiled binary

We are going to use two tools that should be available with either your normal binutils installation (which you should have), or the one that we downloaded above (arm-none-eabi-binutils). With these tools we can study the binaries we just compiled. Execute the following command:

```
readelf -h babysteps.elf
```

This should spit out a bunch of information from *somewhere in the file*.

Explain what this information means	Points: .5
-------------------------------------	-------------------

We can use readelf to figure out more information by passing another flag:

```
readelf --section-headers babysteps.elf
```

This gives you a description of the sections of the very, very simple binary we just built (it's impressive, the file is 5.4k in my machine, and we just wrote 6 lines of assembly..)

Research on what this information means	Points: .5
---	-------------------

3.2 Disassembling the compiled binary

In theory, we could reverse the compilation process. Let's give it a try:

```
arm-none-eabi-objdump -D babysteps.elf
```

This should spit out a lot of information!

Can you find the code we wrote? where is it? what's the rest of code there?	Points: 1
---	------------------

4 Running ARM binaries we just built

It follows that we want to run our program. So let's give it a try

4.1 Using a CPU emulator

Elaborate on what you did to run it in an emulator, and what happened	Points: 1
---	------------------

4.2 Using a Virtualized machine

Let's put the same program inside of the zynq device we used above. We are going to do a bit of a "hack" to get around the fact that this is an ELF [1] file, rather than a baremetal build (does it matter?).

```
qemu-system-arm -machine [your guess here] -kernel babysteps.elf
```

We are using a USBArmory MKII board, not a zynq-a9. What would be the closest machine we could use for our emulator? **Points: 2**

Once this runs, nothing probably happens. This is ok, our code really does nothing (it's the fastest zero-number-generator ever written, in fact), so it'd be hard to figure out what exactly is happening.

Do notice, however, that in the qemu window you can check three different views: `compatmonitor0` (somewhat of an admin interface), `serial0`, which is the serial port output (this will be useful in the future), and `parallel0` a parallel port to do the same. You can configure qemu to connect to different bits of the board by passing further flags (which we will do, to read a second serial port). For now, instead, we want to see the code as its being loaded.

Extra credit: modify the assembly code to print a string to the serial0 port upon startup **Points: 1**

5 Debugging our environment

Finally, it'd be fantastic to know something is going on, but we don't have the niceties of a regular debugging environment (not even `printf`), so we need to resort to lower-level methods.

Using the methods from Section 2, download another tool `arm-none-eabi-gdb`. This is our debugging tool for our virtual environments. Needless to say, if you get stuck here, reach out to me and we'll figure it out.

Having installed `gdb`, we will use `qemu-arm` to run out binary and debug it.

```
qemu-arm -singlestep -g 1234 babysteps.elf
```

The program should hang (as it should), but it should be ready for us to connect to it. Open another terminal and type:

```
arm-none-eabi-gdb
```

Which should drop you into a shell (I'm copy-pasting from my side)

```
[santiago@hostname workbench]\$ arm-none-eabi-gdb
GNU gdb (GDB) 13.2
...
Type "apropos word" to search for commands related to "word".
(gdb) file babysteps.elf
...
(gdb) target remote localhost 1234
Remote debugging using localhost:1234
_start () at babysteps.s:4
```

Things seem to be working if you were dropped to the current instruction. To go over what we did, we run the CPU emulator, we told it to wait for our debugger, and connected to it from the other side through a socket. Using this, we are able to slowly execute and study our program. Let's give it a try:

```
(gdb) b babysteps.s:4
Breakpoint 1 at 0x8000: file 2/strap2.s, line 4.
(gdb) c
Continuing.
```

```
Breakpoint 1, _start () at 2/strap2.s:4
4          eor r1, r1
```

We used two commands, `b` to create a breakpoint, `c` to run the program. We ran into the breakpoint we set (which was where we started). You can use other commands such as `list/l` or `si` to inspect the code a little bit more.

Elaborate on what we did and why

Points: 1

6 Modifying our basic code

For now, this code is useless, and we wanted to make a memcheck. Given that we've covered a lot of territory, it'd probably be better to not make a full fledged memory checker. Instead, let's see if the memory we loaded is all actually 0. Consider the following pseudocode:

```
int *base = FLOOR
value = 0
for some_range_you_decide:
    value = next_memory_chunk
    if value != 0:
        explode()
```

This looks daunting, but consider the tools you've got. For example, you can add a breakpoint on `explode()` to check if there are "memory errors" I uploaded an arm reference to brightspace, but, if you want to go down the easy path you can build using `gcc` and `as/ld`. Hell, if you want to use `go` or `rust`, be my guest (or reach out to me to help you get started)

Share the code of your implementation and explain it
--

Points: 1

7 Closing thoughts

I hope this demystifies how a lot of the tools you've used during your career work. This is just the building block to start building a better understanding of the stack we are going to be working with. Good luck!

Extra credit Please share how hard/easy this was for you. I want to adjust workloads/labs based on the difficulty of this first attempt
--

Points: 2

References

- [1] elf - format of executable and linking format (elf) files. <https://www.man7.org/linux/man-pages/man5/elf.5.html>.

- [2] A generic and open source machine emulator and virtualizer. <https://www.qemu.org>.
- [3] Gnu grub. <https://www.gnu.org/software/grub/>.